Vue 3 Rendering Mechanism — Шпаргалка

Три стадии пайплайна

```
Шаблон (.vue)
  ↓  компилятор (build time)
render-функция  →  vnode-дерево
  ↓  runtime (mount)
Реальный DOM  +  reactive effect (отслеживание зависимостей)
  ↓  при изменении зависимости (patch)
Новое vnode-дерево  →  diff  →  минимальный DOM update
```

Стадия	Что происходит	Зачем / Средства
Compilation	Шаблон → render-функция	Компилятор **анализирует шаблон статически** и добавляет в сгенерированный код числовые флаги, метки блоков и хойстинг статики. Это подсказки для runtime — без них он работал бы вслепую и сканировал всё подряд. Средства: `createElementVNode`, `openBlock`, `createBlock` с флагами.
Mounting	render-функция запускается первый раз, строится реальный DOM	Функция выполняется внутри `watchEffect`-подобного механизма — любое обращение к реактивному значению **автоматически регистрирует зависимость**. Так Vue знает, какие данные влияют на какой компонент.
Patching	При изменении dep — перезапуск, diff, точечный DOM update	Перезапускается только render-функция **конкретного компонента** (не всего дерева). Diff идёт не по всему vnode-дереву, а только по плоскому списку динамических узлов блока — благодаря подсказкам из шага 1.

Что такое «подсказки компилятора»

Vue — редкий случай, когда **один автор контролирует и компилятор, и runtime**. Это позволяет при сборке вшить в сгенерированный JS-код метаинформацию, которую runtime использует вместо универсального обхода:

Подсказка	Как выглядит в коде	Что говорит runtime

Подсказка	Как выглядит в коде	Что говорит runtime
Patch Flag	число 4-м аргументом в `createElementVNode(..., 2)`	«у этого элемента может меняться только `class`»
Block marker	`openBlock()` + `createBlock()` вместо обычного вызова	«ниже стабильная структура — запомни только динамических потомков»
Hoisted ref	`const _hoisted = createElementVNode(...)` вне render-функции	«этот узел статичен — не пересоздавай и не диффи»

Без этих подсказок runtime должен был бы при каждом патче **сравнивать всё** — именно так работает React.

Vnode — что это

Просто JS-объект, описывающий узел UI:

```
const vnode = {
  type: "div",
  props: { id: "hello", class: "box" },
  children: [
    /* вложенные vnode */
  ],
};
```

Renderer обходит дерево таких объектов и применяет изменения к реальному DOM.

Оптимизации компилятора

Static Hoisting — подъём статики

Узлы без динамических привязок создаются **один раз** и кэшируются. Runtime пропускает их при диффинге (сравнение ссылки → пропуск).

```
<div>
  <span>Статика</span>
  <!-- создан раз, больше не трогается -->
  <span>{{ dynamic }}</span>
  <!-- пересчитывается при изменении -->
</div>
```

Несколько статичных элементов подряд → сливаются в один vnode через `innerHTML`.

Patch Flags — флаги патча

Компилятор **заранее** знает, что может измениться у элемента, и кодирует это числом прямо в вызове `createElementVNode`:

```
// скомпилированный код
createElementVNode(
  "div",
  { class: normalizeClass({ active: ctx.active }) },
  null,
  2 /* CLASS */,
);
//
↑
//
флаг: менять только class
```

Runtime проверяет флаг через **битовую операцию** — не сканирует все пропсы.

Флаг	Что обновлять
1 TEXT	текстовый контент
2 CLASS	атрибут `class`
4 STYLE	атрибут `style`
8 PROPS	конкретные пропсы (список в vnode)
64 FULL_PROPS	все пропсы (динамические ключи)

Block Tree — блоковое дерево

Шаблон разбивается на **блоки** — структурно стабильные секции. Каждый блок хранит **плоский массив** только динамических потомков.

```
Root Block
  ├─ <div :id="x">        ← patch flag PROPS
  └─ <p>{{ msg }}</p>     ← patch flag TEXT
```

При патчинге runtime обходит только этот плоский список — не всё дерево.

- `v-if` / `v-for` → создают **вложенные блоки**, изолируя структурные изменения внутри себя.

Vue vs React — честное сравнение

Архитектура reconciler

React использует **Fiber** — внутреннее представление дерева, где каждый компонент это «волокно» (fiber node). Fiber заменил старый Stack Reconciler в React 16.

Концепция	Старый Stack (до React 16)	Fiber (React 16+)
Выполнение	Синхронно, один большой рекурсивный обход	Разбито на единицы работы (work units)
Прерываемость	Нет — блокирует main thread до конца	Да — можно приостановить и продолжить
Приоритеты	Нет	Есть — срочные обновления (клик, ввод) прерывают фоновые
Диффинг	Тот же алгоритм	Тот же алгоритм, но исполнение асинхронное

> Fiber изменил **когда и как** запускается работа — но не **что именно** сравнивается. Алгоритм диффинга остался прежним.

Что React 18 добавил поверх Fiber

- **Concurrent rendering** — рендер прерываем, React может бросить незаконченное дерево и начать заново при срочном обновлении
- **Automatic batching** — несколько `setState` в одном таймере/промисе теперь автоматически объединяются в один рендер (раньше только внутри React-событий)
- `startTransition` — явная пометка «это обновление несрочное, можно прерывать»

Сравнение с Vue 3

Аспект	Vue 3	React (Fiber + 18)
Архитектура	Компилятор + runtime в связке	Только runtime (Fiber scheduler)
Единица работы при ре-рендере	Компонент целиком, но diff только по динамическим узлам блока	Компонент + все потомки вниз (если нет `memo`)
Статика	Хойстится при сборке, никогда не диффится	Пересоздаётся как новые объекты каждый рендер
Что именно обновить	Patch Flags — закодировано **при сборке**	Fiber сравнивает все пропсы **в рантайме**
Прерываемость рендера	Нет (синхронный)	Да — Concurrent Mode (React 18)
Приоритизация обновлений	Нет	Да — `startTransition`, `useDeferredValue`
Ручной контроль ре-рендеров	Не нужен	`memo`, `useMemo`, `useCallback` — ты обязан подсказывать

Где React обгоняет Vue

- **Прерываемый рендер** — при тяжёлых обновлениях React 18 не заморозит UI, Vue заморозит
- **Приоритеты** — `startTransition` позволяет явно разделить срочные и фоновые обновления
- **Гибкость** — JSX это просто JS, любая динамика без ограничений шаблона

Где Vue обгоняет React

- **Объём диффинга** — Vue обходит только плоский список динамических узлов, React обходит всё поддерево
- **Статика** — Vue не трогает её вообще, React пересоздаёт vnode-объекты
- **Нет нужды в мемоизации** — компилятор уже знает что статично, React этого не знает

Почему React всё ещё требует `memo` / `useMemo`

Fiber умеет **откладывать и прерывать** работу — но не умеет **не делать** её для неизменившихся поддеревьев без подсказки разработчика. Без `memo` компонент перезапускается при каждом рендере родителя, создавая новые vnode-объекты даже для статичного содержимого.

Vue знает это **заранее** — компилятор вшивает подсказки при сборке.

> **Коротко:** Vue выигрывает меньшим объёмом работы за счёт компилятора. React выигрывает гибкостью и управлением приоритетами через Fiber/Concurrent Mode — но платит за это тем, что ты сам управляешь мемоизацией.

Шаблоны vs Render-функции

```
// render-функция — максимальная гибкость, но нет оптимизаций компилятора
import { h } from "vue";

export default {
  render() {
    return h("div", { class: "box" }, [h("span", "текст")]);
  },
};
```

	Шаблоны	Render-функции
Оптимизации компилятора	Все (hoisting, patch flags, block tree)	Нет
Гибкость	Ограничена синтаксисом	Полная (любой JS)
Читаемость	Высокая	Ниже при сложной логике
Когда использовать	По умолчанию	Переиспользуемые компоненты с динамической структурой